SSARMA (1,2) - Financial Econometrics

Elisabet Cueli Alba

2025-11-16

```r
library(FKF)
set.seed(123)
```

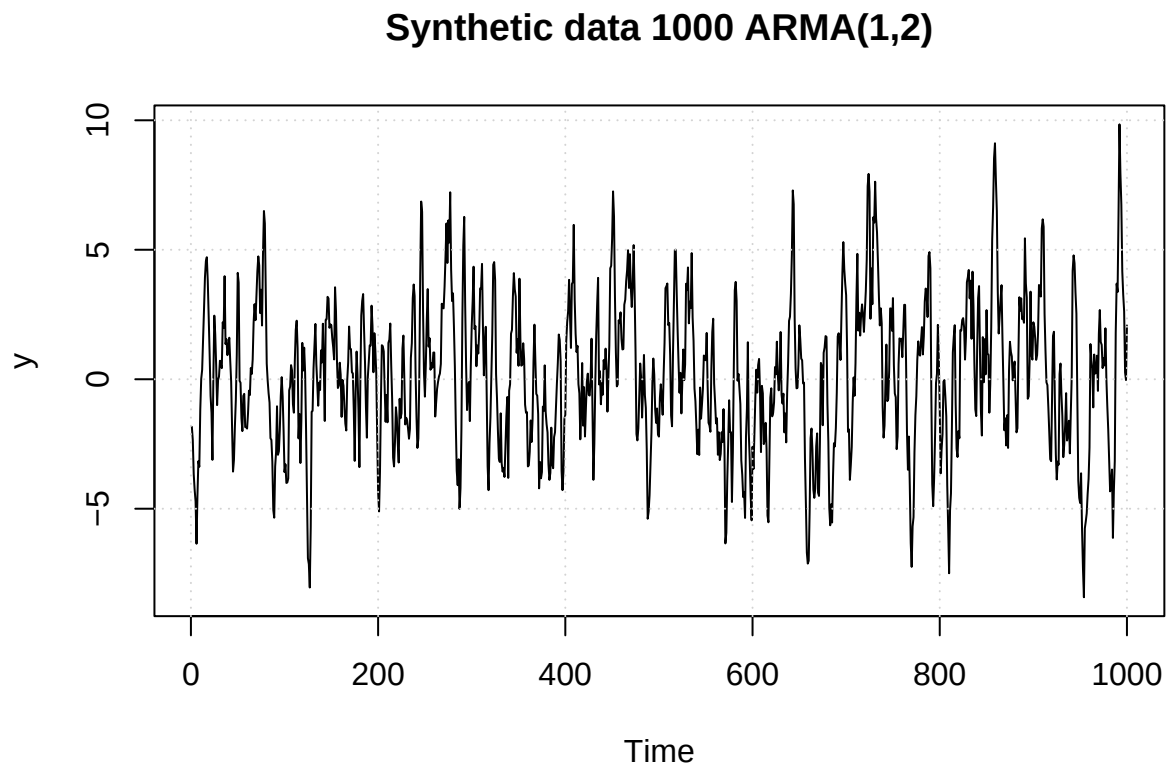
1. Generate Synthetic ARMA(1,2) Data

We generate 1000 observations from an ARMA(1,2) process with parameters: $\phi_1 = 0.7$, $\theta_1 = 0.4$, $\theta_2 = 0.3$, $\sigma = \sqrt{2}$

```r
# True parameters
ar1 <- 0.7
ma1 <- 0.4
ma2 <- 0.3
sigma <- sqrt(2)

# Generate ARMA(1,2) data
n <- 1000
arma_model <- list(ar = ar1, ma = c(ma1, ma2))
y <- arima.sim(model = arma_model, n = n, innov = sigma*rnorm(n))

# Plot the series
plot(y, main = "Synthetic data 1000 ARMA(1,2)")
grid()
```



2. State Equation

The ARMA(1,2) model can be written as:

$$Y_t = \phi_1 Y_{t-1} + W_t + \theta_1 W_{t-1} + \theta_2 W_{t-2}$$

The **state equation** is:

$$\alpha_t = T_t \alpha_{t-1} + \sigma_W \eta_t$$

Where:

$$T_t = \begin{bmatrix} \phi_1 & \phi_2 = 0 & \phi_3 = 0 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}, \quad \sigma_W \eta_t = \begin{bmatrix} W_t \\ 0 \\ 0 \end{bmatrix}$$

Thus, the state equation is of this form:

$$\begin{bmatrix} \alpha_{t,1} \\ \alpha_{t,2} \\ \alpha_{t,3} \end{bmatrix} = \begin{bmatrix} \phi_1 & 0 & 0 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} \alpha_{t-1,1} \\ \alpha_{t-1,2} \\ \alpha_{t-1,3} \end{bmatrix} + \begin{bmatrix} W_t \\ 0 \\ 0 \end{bmatrix}$$

3. Observation Equation

Letting $r \equiv \max\{p, q+1\}$, r=3 in our case (ARMA(1,2)). Given no constant and $G_t = 0$, the **observation equation** is:

$$Y_t = C_t + Z_t \alpha_t + G_t \epsilon_t = Z_t \alpha_t = Z_t \begin{bmatrix} \alpha_{t,1} \\ \alpha_{t,2} \\ \alpha_{t,3} \end{bmatrix}$$

Where:

$$Z_t = \begin{bmatrix} 1 & \theta_1 & ... & \theta_{r-1} \end{bmatrix} = \begin{bmatrix} 1 & \theta_1 & \theta_2 \end{bmatrix}$$

4. Function arma12ss()

This function creates the state-space representation matrices for an ARMA(1,2) model.

```
arma12ss <- function(ar1, ma1, ma2, sigma) {
  ## initial value of the state variable:
  r <- 3
  a0 <- c(0, 0, 0)
  P0 <- matrix(1e6, nrow=r, ncol=r) # MSE (diffuse prior)

  ## State equation:
  dt <- matrix(0, nrow = r)
  Tt <- matrix(c(ar1,0,0, 1,0,0, 0,1,0), ncol=r) # t(T) TRANSPOSE
  Ht <- sigma * matrix(c(1, 0, 0, 0, 0, 0, 0, 0, 0), ncol=r)
  HHt <- Ht %*% t(Ht)

  ## Observation equation:
  ct <- matrix(0)
  Zt <- matrix(c(1, ma1, ma2), ncol=3)
  GGt <- matrix(0)

  return(list(a0 = a0, P0 = P0,
              dt = dt, ct = ct,
              Tt = Tt, Zt = Zt,
              HHt = HHt, GGt = GGt))
 }
```

5. Objective Function for Maximum Likelihood

We create a function that returns the negative log-likelihood to be minimized.

```
objective <- function(param,yt){
  #SS representation of ARMA(1,2)
  ss <- arma12ss(
    ar1=param["ar1"],
    ma1=param["ma1"],
    ma2=param["ma2"],
    sigma=param["sigma"])

  ans <- fkf(a0 = ss$a0, P0 = ss$P0, #output of arma12 is input for fkf
             dt = ss$dt, ct = ss$ct,
             Tt = ss$Tt, Zt = ss$Zt,
             HHt = ss$HHt, GGt = ss$GGt,
             yt = yt)

  return(-ans$logLik)
}
```

6. Parameter Estimation using optim()

Estimation is then performed using a numeric search by optim {stats}.

```
param0 <- c(ar1=0, ma1=0, ma2=0, sigma=1)

 fit <- optim(par = param0, # initial values
```

```r
              fn = objective, # logLik to be maximized
              yt = rbind(y), # (1,1000) to be passed to "fn"
              hessian = TRUE)
names(fit)
```

```
## [1] "par"         "value"       "counts"      "convergence" "message"
## [6] "hessian"
```

```r
fit$par
```

```
##        ar1        ma1        ma2      sigma
##  0.8504149 -2.1989328 -4.3470317  1.5015560
```

7. Construct 95% Confidence Intervals

We compute standard errors from the Hessian matrix and construct 95% confidence intervals for each parameter.

```r
p <- cbind(
  actual=c(ar1=ar1, ma1=ma1, ma2=ma2, sigma=sigma),
  estimate=fit$par,
  lowerCI = fit$par - qnorm(0.975)*sqrt(diag(solve(fit$hessian))),
  upperCI = fit$par + qnorm(0.975)*sqrt(diag(solve(fit$hessian)))
)
p
```

```
##         actual   estimate     lowerCI   upperCI
## ar1   0.700000  0.8504149   0.8177491 0.8830807
## ma1   0.400000 -2.1989328  -9.5886675 5.1908018
## ma2   0.300000 -4.3470317 -14.1590484 5.4649849
## sigma 1.414214  1.5015560   1.4356546 1.5674573
```

Conclusion

Only actual ar1 and sigma seems to fall out of the Confidence Interval, but the CI for ma1 and ma2 seem statistical unsignificant as they are too wide.

8. Filter the Series using fkf()

We apply the Kalman filter with the estimated parameters to obtain filtered state estimates.

```r
sp<- arma12ss(
  fit$par["ar1"], fit$par["ma1"],fit$par["ma2"], fit$par["sigma"]
)

names(sp)
```

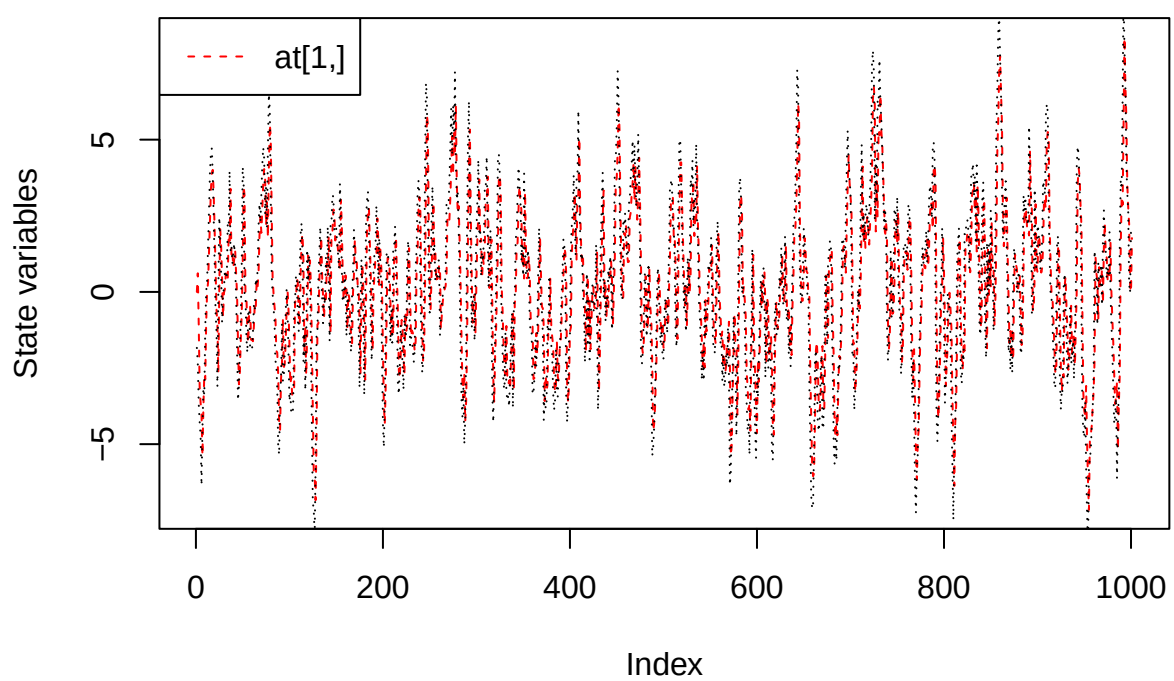
```
## [1] "a0"  "P0"  "dt"  "ct"  "Tt"  "Zt"  "HHt" "GGt"
```

```r
ans <- fkf(
  a0 = sp$a0,
  P0 = sp$P0,
  dt = sp$dt,
  ct = sp$ct,
  Tt = sp$Tt,
  Zt = sp$Zt,
  HHt = sp$HHt,
  GGt = sp$GGt,
  yt = rbind(y)
)
names(ans)
```

```
##  [1] "att"      "at"       "Ptt"      "Pt"       "vt"       "Ft"
##  [7] "Ftinv"    "Kt"       "logLik"   "status"   "sys.time" "yt"
## [13] "Zt"       "Tt"
```

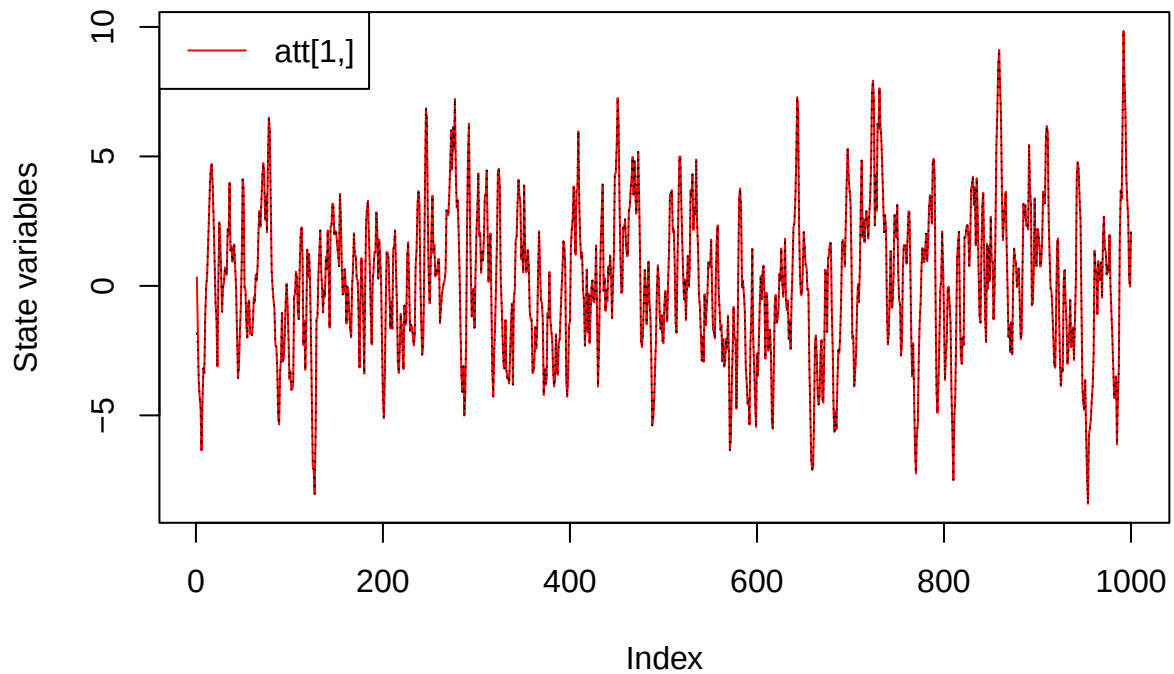
9. Plot Predicted Series vs Realization

```r
plot(ans,at.idx=1,att.idx=NA, CI=NA)
lines(as.vector(y), lty ="dotted")
```



10. Plot Updated Series vs Realization

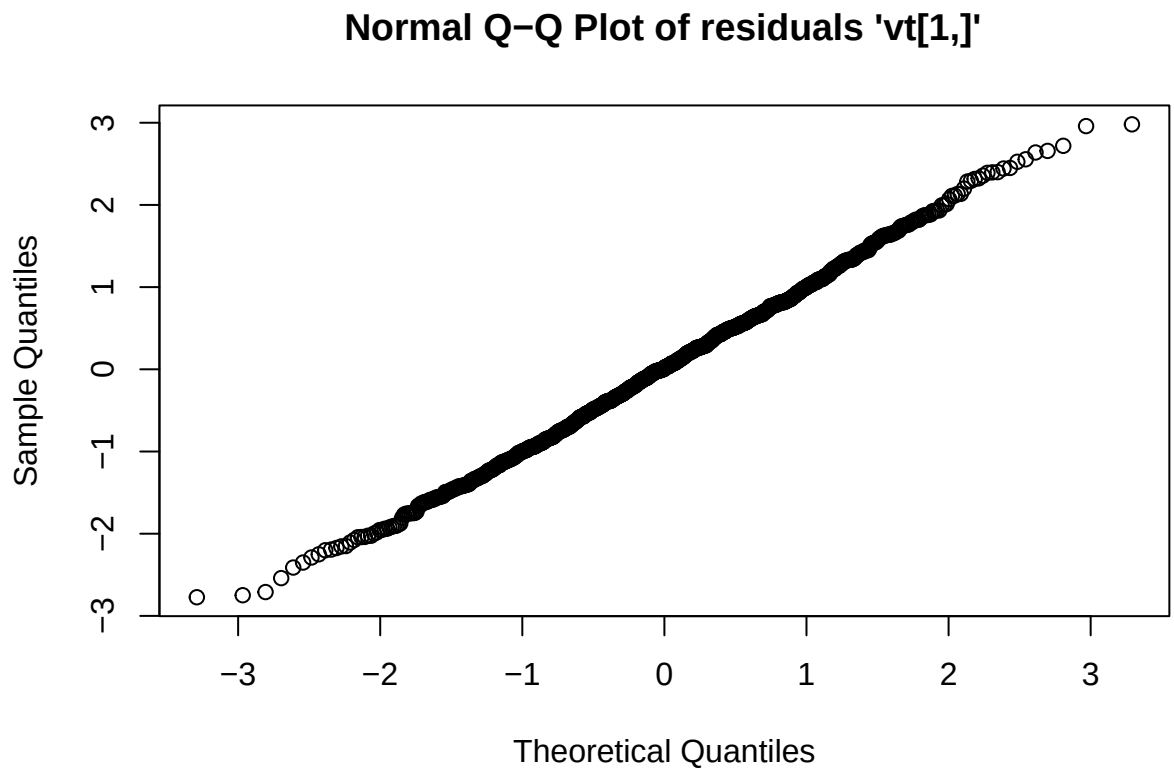
```r
plot(ans,at.idx=NA,att.idx=1, CI=NA)
lines(as.vector(y), lty ="dotted")
```



11. Diagnostic Plots: QQ and ACF of Residuals

We examine the residuals to verify the normality and independence assumptions.

```r
plot(ans,type="resid.qq")
```



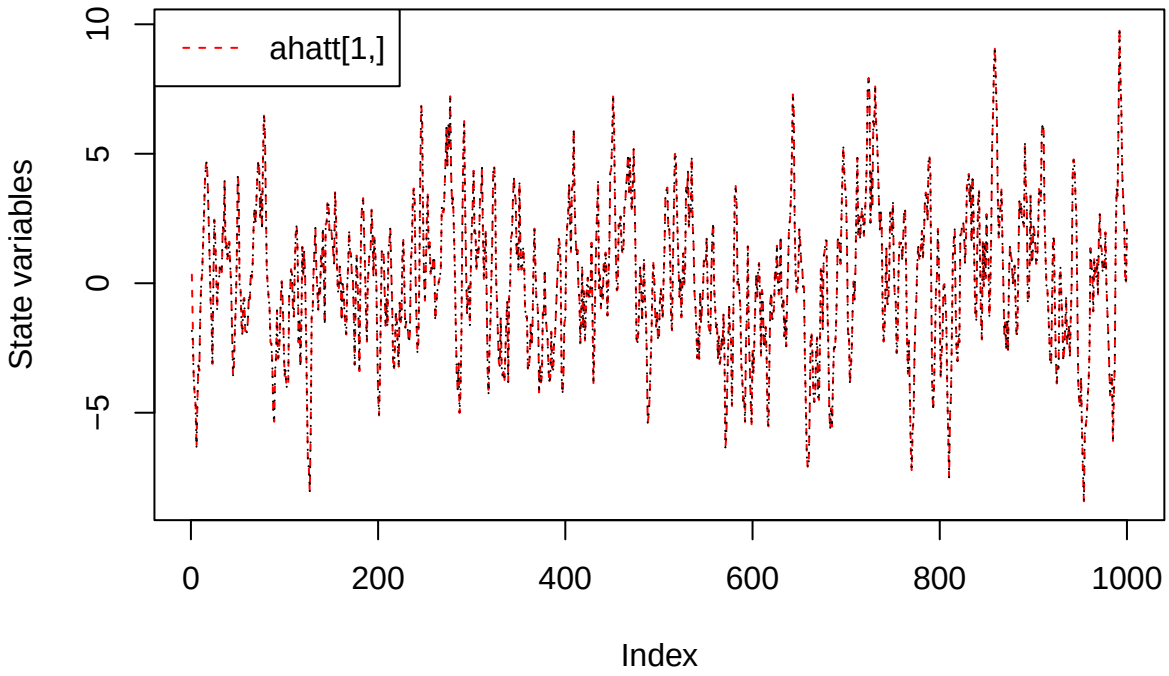
12. Smoothed Estimates using fks()

The Kalman smoother produces $a_{t|n}$, which uses all available information from the entire sample.

```
sm <- fks(ans)
names(sm)
```

```
## [1] "ahatt" "Vt"
```

```
plot(sm,CI=NA,ahatt.idx = 1)
lines(as.vector(y),lty="dotted")
```



Conclusion

All forecasted, updated and smoothed series are very closed to the realized series, denoting a good fit for the model.

Regarding the Q-Q graphic, the body of the distribution is well alike to the Normal Distribution, though with a slight deviation in the tails being heavier, meaning higher probabilities for both negative and positive outliers, characteristic that resambles a T-Student distribution.